# Support it! A Dynamic Approach to Application Support for Research OSes

Hugo Lefeuvre[1], Gaulthier Gain[2], Vlad-Andrei Bădoiu[3], Daniel Dinca[3], Costin Raiciu[3,4], Pierre Olivier[1], Felipe Huici[5,6]

*[1]The University of Manchester, [2]University of Liège, [3]Politehnica Bucharest, [4]Correct Networks, [5]Unikraft.io, [6]NEC Labs Europe*

**EuroSys'22**
*Rennes, France*

Research Supported by:

Horizon 2020 European Union Funding for Research & Innovation

UK Research and Innovation

## Compatibility: The Crux of Research OS Implementation

- There is a **need for new research OSes**: to advance the state of the art in systems software, cope with new hardware trends, meet new applications demands.
- Unfortunately, developing research OSes is widely considered as a **challenging task.**
- Since 2011, only 36 out of 1293 papers in top OS conferences (SOSP, OSDI, EuroSys, ASPLOS) proposed a brand-new OS.

- **Compatibility with existing applications** is at the heart of this complexity.
- Why? **Supporting popular software is necessary** to evaluate a research OS, to build a community, and transition to the industry.

**But what does *supporting* mean?**

- **Supporting Linux apps is mainly implementing Linux system calls**.
- Past research came to discouraging conclusions: **one must implement 100s of system calls** to support applications [1].

## Which OS Features Do You (really) Need to Implement in Your OS Prototype?

- In practice, **the requirements on system call implementation are not as stringent** as previous studies make them look.
- Previous work **overestimated the required set of system calls** because they 1) focused on an **overly broad panel of applications**, 2) used binary static analysis, resulting in **false positives**, and 3) adopted an **overly strict definition of support**.

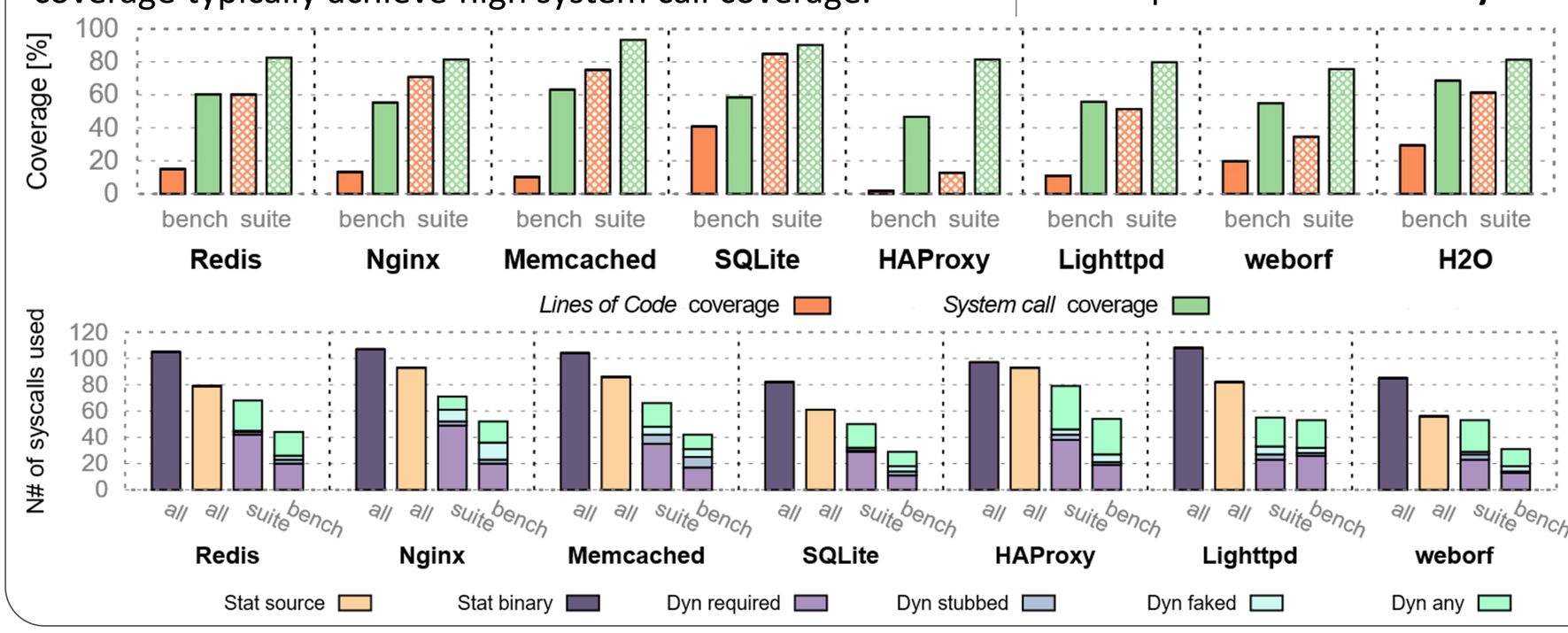
- In this work, we aim at measuring the system call usage of applications in a way that is the **closest possible to what OS designers need to implement** to make the application run.
- The goal is to **limit and guide the development effort** of a new research OS that aims at supporting Linux applications.
- We avoid the limitations of past works by relying on **dynamic analysis on a curated set of applications.**

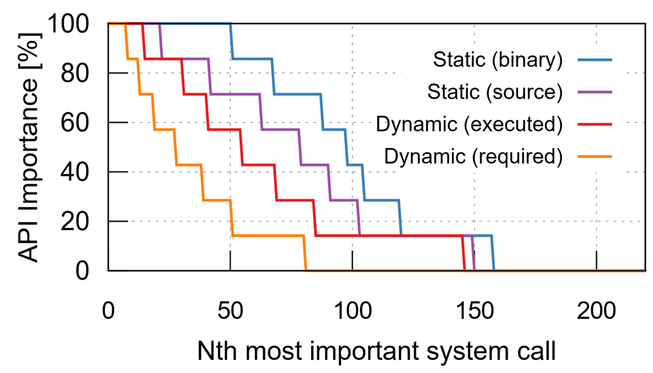
## Dynamic Analysis Can be Precise, Static Analysis is *Very* Pessimistic

**What is a *comprehensive workload*?** While application benchmarks (*e.g* `wrk`) feature relatively low syscall coverage (> 18%), **test suites are very effective at covering application system calls** (> 80%). Even suites with low line coverage typically achieve high system call coverage.

We developed a dynamic analysis framework using seccomp and ptrace, and applied it to a set of popular cloud applications.

- We observe a **large difference between static and dynamic analysis**.
- Binary **static analysis reports 80-110 syscalls** required for each app, compared to **50-80 for dynamic analysis** with test-suites.




- *Top left*: LoC (orange) and syscall (green) coverage for benchmarks (bench) and test suites (suite)
- *Bottom left*: System call usage measurement with static binary, static source, and dynamic analysis
- *Top right*: API importance with the different types of analysis

## Applications are Resilient to Partial Implementation

In addition to measuring system call usage, our tool can detect which system calls can be **stubbed** by the OS (don't implement and return `-ENOSYS`) and **faked** (don't implement and return success code). These practices are very common among OS developers.

- Our findings show that **a large portion of the system call API can be reliably stubbed or faked**: as much as 46% of system calls for test suites.
- We observe that system calls can be stubbed or faked mainly because 1) they are optional, e.g., the application defaults to a sane value in case of failure, 2) they are alternatives (brk vs. mmap), or 3) their failure is non-critical.

## Conclusion

As much as 60% of system calls identified as needed by previous works are not required for those applications to pass their official test suites. Yes, developing OSes is hard, compatibility is hard, but probably not as hard as one might think. Provided a methodic approach is taken, the majority of the work lies in core OS features. This is a message of hope; develop research OSes!

## References

[1] Tsai et. al. *A study of modern Linux API usage and compatibility: what to support when you're supporting*. EuroSys'16.